Ailo -- Product Requirements Document (v3)

Ailo (pronounced "EYE-low": AI + allowance) is the operating system for agent money.

Give your AI an allowance, not your bank login.

Version 3.1 -- 23 Oct 2026 (stack, team and Kora corrections added 9:30 PM) -- PAU 24-Hour Hackathon (build window Fri 2:00 PM to Sat 2:00 PM). Targets: AGENTS + TRUST + MONEY. Ecosystem name: **Ailo**. Feature names are working names.

Source rule: every architecture choice below copies a documented production pattern, cited inline with its source in `stages/01-research/output/production-precedents.md` (P1-P10). Anything without a precedent is marked **[our design]**. Regulatory facts we could not verify are marked **[unverified]**.

---

1. Summary

AI has moved from answers, to solutions, to actions, and now to outcomes. Outcomes cost money, so agents must pay, get paid, prove who they are, and answer for what they spend. The financial system and the web were built for a human clicking buttons.

Ailo is the operating system for agent money: the rails that let builders create any role in the agent economy (the agent that pays, the business that gets paid, and the financial services in between), with every naira carrying a record of why it moved.

Ailo holds no money. Funds stay with a licensed partner (the Kora sandbox for the hackathon). Ailo decides whether an agent's payment instruction may reach the partner, and keeps a mirror ledger reconciled against the partner. (Pattern: a platform instructs a licensed holder, as in Stripe Connect separate charges and transfers, P6.)

2. Problem

1. **Agents cannot be trusted with money.** Letting an agent pay today means handing it a card, a bank login or a provider's secret key: no limits, no audit, no off switch. Prompt injection is unsolved at the model level. OWASP, OpenAI and Anthropic all say to enforce limits outside the model (P10).
2. **Merchants cannot trust agents.** "Until this exists, merchants will keep blocking agents at the firewall." -- Sean Neville, a16z Big Ideas 2026.
3. **The web is built for humans.** Checkouts and OTP screens assume a person.
4. **No one can explain where agent money went.** There is no rule, reason or evidence trail, so there is no recourse.
5. **Global answers skip Nigeria.** AP2, x402, Stripe ACP, Visa TAP and Mastercard Agent Pay do not settle in Naira on NIP. Paystack Index is a closed catalogue (landscape.md).

"The bottleneck for the agent economy is shifting from intelligence to identity." -- Sean Neville

3. The OS and its capabilities

Capability	Working name	What it does	Production pattern copied
Agent identity	**Badge**	The owner signs a mandate containing the agent's public key. The agent signs every request with a short expiry and nonce. Payees verify against a published JWKS	AP2 open/closed mandates with `cnf` (P7). Cloudflare Web Bot Auth / RFC 9421 Ed25519 (P7). Skyfire `aud` + `jti` checks (P7)
Spend rules	**Spec Gate**	The owner writes a rule in English. The LLM proposes it in our rule syntax. The owner tests it against past payments and saves it. Only the saved deterministic rule is enforced	Stripe Radar Assistant: prompt, then rule, then Test, then explicit Add (P2)

Risk second opinion	**Second Look**	An LLM intent check on flagged payments, with untrusted content marked. It can only escalate or block, never approve	Radar ML score feeds deterministic rules; "review" is a human queue (P1). Spotlighting as one layer only (P10)
Owner approval	**Nod**	A paused server-side state. Only the authenticated owner decides, checked against the stored pending item. Approval issues a one-time authorization capped at the amount, with an expiry	OpenAI Agents SDK `needs_approval` / interruptions (P9). ACP shared payment token `max_amount` + `expires_at` (P9)
Kill switch	**Brake**	Revokes the mandate and the agent's key, and cancels pending approvals and holds	Stripe card/cardholder status controls (P1) **[our composition]**
Accept payments	**Till**	Any site or API answers agents with 402 + price. The paid retry carries a signed authorization (nonce + validity window). The agent can send a max-price header	x402 exact scheme + nonce replay protection (P8). Cloudflare `crawler-max-price` / `crawler-charged` (P8)
Escrow	**Hold**	A pending transfer with an expiry. It releases when the buyer accepts or the inspection window lapses after verified delivery. Rejection inside the window opens a dispute	TigerBeetle two-phase transfers (P5). Escrow.com inspection period, Upwork 14-day auto-release (P6)
Ledger + reconcile	**Square Up**	A double-entry mirror ledger, pending/posted, reversals only. The reconcile button compares it with Kora	Modern Treasury pending/posted + balance filters (P5). Kora verify-by-reference (P3, P4)
Decision trace	**Receipt**	Every payment stores its rule version, the agent's reason, check results, the verdict and the Kora reference	Radar rule history audit trail (P2) **[trace format: our design]**
Services marketplace	**Exchange**	Insurers, lenders and data APIs plug in (roadmap)	--

4. Users and roles

Role	Who	Job to be done
Owner	An individual or SME owner	"Let my agent spend for me without risking my account"
Agent builder	Developers and fintechs building agents	"Give my agent a safe way to pay, in one SDK"
Payee	Shops, APIs, data providers, other agents	"Get paid by agents and know who is paying"
Service provider	Insurers, lenders, data APIs	"Serve agents using real risk data" (roadmap)
Operator	A partner bank or fintech ops team	"Monitor agents, reconcile, resolve disputes"

5. Architecture decisions (each one sourced)

5.1 Money custody and funding. The owner tops up a Kora NGN virtual account (sandbox credit endpoint), which books `partner_cash` against `owner_wallet`. Ailo never holds funds. The demo balance is labelled "sandbox only". The claim that only MMOs, not PSSPs, may hold customer funds is **[unverified]**, so the pitch says "funds stay with the licensed partner" and does not cite a licence rule.

5.2 Check order for every payment (Stripe Issuing runs spending controls before the real-time hook, P1):

1. **Signature + key scope.** Is the request signed by the agent key in the mandate, not expired, nonce unused (P7)?
2. **Deterministic rules (Spec Gate).** Caps per interval and category in kobo (Stripe `spending_limits` shape, P1), payee allowlist, expiry, velocity. **An unknown payee is never sent to the AI: it goes to Nod or is declined.**
3. **Reserve.** Create a pending hold under a balance condition (`available >= 0`) so concurrent payments cannot both pass a cap (Modern Treasury balance filter / TigerBeetle pending, P5).
4. **Second Look**, only on flags (first payment to a payee today, an unusual category for the stated goal, a payee-page content trigger). It can only add friction (escalate to Nod or block), never approve (Radar: ML is an input and "review" is a human queue, P1; OWASP least privilege, P10).
5. **Hard deadline.** If Second Look doesn't answer within 2 s, the default is **escalate to Nod** (Stripe 2 s /

Marqeta 3 s timeout with a default decision, P1). Clean payments never wait on a model.
6. **Nod** if over the threshold, flagged or a new payee (P9).
7. **Submit to Kora** via an outbox step, outside the DB transaction.

Splitting payments. The approval threshold applies to a rolling 24h total per payee, plus per-payee rate limits (Stripe interval-based limits, P1).

5.3 Rule compilation (Spec Gate). The LLM outputs a `SpendPolicy` (`packages/core`), shaped like Stripe Issuing `spending_controls` (P1): allowed categories, allowed payees, `spending_limits[{amount_kobo, interval}]`, Nod thresholds and an expiry. It is validated by zod and evaluated by plain code (no rules engine). The owner sees the English and the compiled rule side by side, clicks **Test** (replays the last N demo payments and shows which would be blocked), then **Save**. Only saved rules are enforced (Radar Assistant, P2).

5.4 Idempotency. `Idempotency-Key` on every agent POST. Store (key, request hash, status, response) when execution starts. Replay the stored response on a match, error on a parameter mismatch, and return a retryable "in progress" error on a concurrent duplicate (Stripe, Adyen, P3). Toward Kora, `reference = AL-{payment_id}`, **fixed for the life of the payment**. Every retry, including after a timeout or 5xx, reuses the same reference. On "Duplicate Transaction Reference" or a 5xx, **query by reference, never re-send with a new reference** (Kora docs, P3). A new reference (`AL-{payment_id}-r{n}`) is allowed only after a query confirms a **final** failure on the previous one, so a timeout can never produce a second payout. Kora `metadata` allows at most 5 keys with **no underscores** in key names (e.g. `paymentid`, not `payment_id`). Kora returns amounts as strings: convert with `nairaToKobo`.

5.5 Payout lifecycle. States: `requested -> blocked | held -> pending_approval -> approved -> submitting -> submitted -> settled | failed | unknown -> (reconciled) | reversed | expired | cancelled | disputed | refunded`. A timeout or 5xx sets `unknown`, keeps the hold, and re-queries. Settled only when Kora's amount, currency and reference match ours; otherwise `needs_review` (Kora: 5xx is not final, P3).

5.6 Webhooks. Verify `x-korapay-signature` (HMAC-SHA256 of `data` with the secret key) on the raw body. Insert `(event, data.reference)` under a UNIQUE constraint (Kora sends no event id). Return 2xx, process async, never rely on order, and treat the webhook as a hint that triggers a re-query (Stripe webhook best practice; Kora webhook docs, P4). Every payout sends its own `notification_url` built from `PUBLIC_BASE_URL`, so local (ngrok static domain) and deployed environments each receive their own webhooks.

5.7 Ledger. Double-entry in kobo. Accounts: `partner_cash`, `owner_wallet`, `holds`, `escrow`, `payee_payable`, `fees`. Holds and escrow are pending transfers with an expiry, posted or voided exactly once, and posted entries are corrected only by reversing entries (TigerBeetle, Modern Treasury, P5).

5.8 Escrow release (Hold). Conditions are fixed at creation. Release when the buyer accepts, or when the inspection window lapses **after verified delivery** (Escrow.com: silence releases only with delivery evidence; Upwork 14-day auto-release, P6). Rejection inside the window moves it to `disputed`. Holds expire (Stripe documents maximum hold windows, P6). Delivery evidence = a payload signed by the payee's key + a schema check + a stored SHA-256 **[our design]**. The AI never releases funds.

5.9 Identity and key scope (Badge). The agent key can only call `get_mandate`, `pay` and `payment_status`. Owner actions (approve, mandates, release, Brake) need an authenticated owner session (OpenAI Agents SDK: authenticate the reviewer, P9). The payout bank account comes from the server-side payee registry, never from a 402 payload **[our design]**.

5.10 Reconcile (Square Up). A button + timed job: query every non-final payment by reference and compare Kora's records with the ledger. The ops console shows "breaks: 0".

5.11 Disputes. States `disputed`, `reversed`, `refunded`. Ops gets a case with the Receipt attached. The Paystack-style accept/reject-with-evidence flow (ops gaps file, URL to verify). The dispute window (48 h vs 72 h) is **[unverified]**: do not quote a number on stage.

6. Hackathon MVP scope

Must (never cut): Spec Gate (compile, test, save), deterministic rules, reserve with a balance condition, Second Look on flags, Nod (phone page), Brake, the goal-driven buyer agent (MCP tools), the demo shop with Till + the poisoned page, Kora sandbox payout + re-query, Receipt view, idempotency replay test, the eval, a one-click demo reset, **Square Up reconcile button** (ops console shows "breaks: 0").

Should (build if the 7 PM checkpoint is met): Hold (escrow), the paid research API (agent pays agent), the webhook handler.

Cut first if behind (in this order): Hold, then the paid research API, then the concurrent webhook test, then the hash-chained trace, then the live ops feed, then MCP over HTTP (fall back to stdio), then the webhook handler (rely on re-query).

Roadmap cards: Exchange (insurance, credit), USSD/WhatsApp approvals, real BVN/NIN verification (the Kora sandbox supports test BVN/NIN), multi-rail connectors (Paystack, Monnify, banks).

Sponsor: mentioned, not integrated. Banks can white-label Ailo as an "agent account".

7. Demo (2 minutes, 5 beats)

1. **Spec Gate (0:00-0:20).** "My shop agent can spend ₦50,000 a week on stock. Ask me above ₦10,000." The compiled rule appears. Test, then Save.
2. **Till (0:20-0:45).** The agent gets the goal "restock rice and oil under ₦20,000", compares two shops, gets a 402, and pays ₦8,500. The ledger ticks. The Kora reference appears.
3. **Second Look (0:45-1:15).** A poisoned product page steers the agent to a payment that **passes every rule** (allowed shop, normal amount, wrong item). Second Look escalates it and the Receipt shows "rules: pass, Second Look: blocked" with the reason. A labelled "compromised agent" toggle guarantees the attempt.
4. **Nod (1:15-1:40).** A ₦12,000 order goes to the phone (mirrored on the projector). Approve. Settled after the Kora re-query.
5. **Brake (1:40-2:00).** Brake, then the next call returns `MANDATE_REVOKED`. Cut to the eval slide.

Live vs mocked is shown on screen. A backup video is recorded by 11 AM Saturday.

8. Evaluation

- 20+ attacks written **blind by a teammate who did not write the rules**, plus 10+ legitimate tasks.
- Report a no-guard baseline, then per tier: blocked by rules / by Second Look / by Nod / missed, plus false blocks. Include the attack classes from `gaps-security.md` (in-allowlist misuse, splitting, velocity, reason-text injection, 402 price swap, replay).
- Agent tests:
- Idempotency replay: the same key sent twice gives exactly 1 payout.
- Weekly cap under concurrency: 10 parallel ₦8k payments to **10 different allowlisted payees** vs a ₦50k weekly cap. Exactly 6 succeed, 4 are declined for the cap, and the ledger balances.
- Per-payee threshold under concurrency: 3 parallel ₦8k payments to **one payee**. Exactly 1 auto-succeeds. The rest cross the rolling ₦10k/24h threshold and go to Nod (section 5.2).
- Crash safety: kill the process mid-payout, then restart and reconcile. Exactly 1 settled, same reference.

9. Interfaces

Method	Path	Caller	Purpose
POST	/v1/mandates	owner	Create from English (compile, test, save)
POST	/v1/mandates/:id/brake	owner	Kill switch
POST	/v1/payments	agent (signed)	Payment request, `Idempotency-Key`
GET	/v1/payments/:id	agent / owner	Status + Receipt
POST	/v1/approvals/:id	owner	Nod: approve or deny
POST	/v1/holds/:id/accept, /reject	buyer owner	Escrow decision
POST	/v1/webhooks/kora	Kora	Signed, deduped, async
POST	/v1/ops/reconcile	operator	Square Up
GET	/.well-known/jwks.json	payees	Badge verification
GET	/v1/events	owner console	Server-Sent Events: `payment.updated`, `approval.created`

MCP tools: `get_mandate`, `pay`, `payment_status`. Till middleware: `till({ price_kobo })`.

10. Business model

A fee per active agent + a per-decision fee, plus a take rate on **settled** volume (Stripe Connect-style platform pricing, ops gaps file). Micro-calls are netted into one daily payout per payee (NIP cost per transfer makes a flat 1% on a ₦50 call unprofitable; transfer cost **[unverified]**). Buyers: agent builders first, then banks and microfinance banks white-labelling "agent accounts". The ask: 3 builder pilots + 1 MFB design partner.

Why not Kora or Paystack? Paystack Index is a closed store. Banks and PSPs won't let one PSP own every agent's identity, so a neutral layer across rails wins distribution. The honest moat today is speed + neutrality.

11. Competition and positioning

Player	What it does	Gap for Nigeria
Google AP2	Signed mandates, rail-agnostic	A protocol only, no Naira rail
Coinbase x402	HTTP 402 payments	USDC only
Stripe ACP / Issuing for agents	Scoped tokens, agent cards	Not for Naira merchants
Cloudflare pay-per-crawl	Signed bots + 402 pricing	Content only, USD
Visa TAP / Mastercard Agent Pay	Agent-recognised card payments	No Nigerian rollout found
Paystack Index	Agent checkout for airtime/data/food	A closed catalogue

Positioning: "Ailo follows AP2-style mandates and x402-style 402 handshakes, and enforces the owner's plain-English rule before money moves. It is the Naira and Know-Your-Agent layer those protocols leave out."

12. Rubric mapping

Criterion	How we earn it
Innovation	The agent-money OS a16z names, built for Naira rails
Relevance & Impact	Agents + trust + money, judged by payments people
Technical Execution	Production patterns cited per component, a live Kora sandbox, concurrency tests
Scalability	A neutral multi-rail layer, a per-agent + per-decision revenue model
Teamwork	Four owned workstreams, visible in commits
Presentation	5 named beats, an AI-only block, Brake
Bonus	Plain English becoming an enforced rule, a live injection caught that the rules couldn't catch

13. Risks

Risk	Mitigation
Kora test keys delayed	Sign up first. A labelled simulated rail adapter behind the same interface
Scope (about 68h of work vs about 52h available)	The cut order in section 6. Core end to end by 7 PM
Claude resists the injection on stage	A labelled "compromised agent" toggle
A regulatory question we can't verify	Say "funds stay with the licensed partner" and quote no unverified numbers
Wi-Fi	The backup video + a local-only mode

14. Workstreams

Owner	Workstream	Where

Emmanuel (backend)	Rails: payments pipeline, reserve/holds, ledger, idempotency, Kora client + re-query, webhooks, Nod, Square Up, SSE events, Till	`apps/api` modules, `packages/till`
Adrien (AI)	Everything agent: Badge (agent identity + JWKS), agent-signature middleware, Brake, Spec Gate + Second Look, the buyer agent + MCP tools, the eval harness	`apps/api` `agents` + `mandates` + `second-look`, `apps/agent`
Josh (frontend)	Console: Live feed, Rules, Receipt drawer, Agents (Badge + Brake), Square Up, Ecosystem map	`apps/console`
Chidi (frontend)	`/nod` phone page, demo shop + poisoned page	`apps/console` `/nod`, `apps/shop`

Milestones (re-planned at 9:30 PM Friday after a late start): one real Kora sandbox payment end to end by **11 PM**. All 5 beats by **6 AM**. Eval, backup video and polish 6-11 AM. Freeze at **12 PM** Saturday. Submit by 1:30 PM.

15. Stack and repository

Chosen on merit and verified against npm/PyPI on 23 Oct 2026 (details: `pau/ailo-planning/research/`).

Layer	Choice
Repo	npm workspaces monorepo: `packages/core` (contracts, money, states), `packages/till`, `apps/api`, `apps/console`, `apps/shop`, `apps/agent`
API	TypeScript, Express 5, feature modules (`*.routes/service/repository/dto/test`), zod 4
Database	MongoDB (official `mongodb` driver, `db/mongo.ts`, in-memory fallback for tests). Ledger reserve is serialized in-process so concurrent payments cannot overspend (shortcut: one API instance; with several, move to an atomic conditional update on a wallet document)
AI (API)	Vercel AI SDK, provider-agnostic: `LLM_PROVIDER=google|openai|anthropic`. Default Gemini: `gemini-3.5-flash-lite` (Second Look, `maxRetries: 0`, 1.8 s timeout), `gemini-3.8-flash` (Spec Gate)
AI (agent)	Python, Pydantic AI (model string `provider:model`) + MCP tools
Identity	jose, Ed25519 signed requests + JWKS
Live updates	Server-Sent Events from the API (no Supabase Realtime)
Frontend	Next.js 16, React Query, MSW 3, Zustand, Tailwind 4, shadcn/ui
Payments	Kora sandbox over REST (no official SDK). Webhook tunnel: ngrok static domain
Secrets	`.env` per app, gitignored; Kora secret only in `apps/api/.env`